SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 **COURSE NAME**: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 10%

QUESTIONS: 05 **Total Marks**:100

Annexure A

Comparative Analysis

Code of Conduct:

I M. Vidhya(192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1.Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Introduction

Both **K-Nearest Neighbour (KNN)** and **Case-Based Learning (CBL)** are instance-based learning methods. They do not necessarily build a complex mathematical model during training. Instead, they retain previous examples and use them to make predictions for new cases.

Consider a **medical diagnosis system** where patient records contain features such as:

- Age
- Blood pressure
- Temperature
- Blood sugar
- Heart rate
- Symptoms

The system can use previous patient cases to predict whether a new patient has a particular disease.

Comparison

Feature	K-Nearest Neighbour (KNN)	Case-Based Learning (CBL)
Basic idea	Finds the K most similar training instances	Finds and reuses similar past cases
Memory usage	Stores most/all training data	Stores cases in a case library
Similarity measure	Usually Euclidean, Manhattan, Minkowski distance	Domain-specific similarity measures can be used
Prediction	Majority vote for classification	Adapts/reuses solution from similar case
Training	Almost no explicit training	Builds and maintains a case base

Feature	K-Nearest Neighbour (KNN)	Case-Based Learning (CBL)
Adaptation	Limited	Strong adaptation through case retrieval and revision
Explanation	Can show nearest patients	Can explain diagnosis using previous cases
Domain knowledge	Less dependent on domain rules	Can incorporate domain-specific knowledge

Medical Example

Suppose a new patient has:

Age = 45, Blood Pressure = 150/95, Blood Sugar = 180, Temperature = 37°C

Using KNN

The system calculates the distance between the new patient and every stored patient.

For Euclidean distance:

Suppose the three nearest patients have diagnoses:

- Patient 1 → Diabetes
- Patient 2 → Diabetes
- Patient 3 → Hypertension

For **K = 3**, the majority class is Diabetes. Therefore:

Predicted diagnosis = Diabetes

Using Case-Based Learning

CBL retrieves a previous patient case that is highly similar to the current patient.

For example:

Previous case: 46 years, high blood pressure, high blood sugar → Diabetes.

The system can reuse the previous diagnosis and adapt it according to the new patient's symptoms and test results.

Memory Usage

KNN generally stores the complete training dataset because it needs the data to calculate distances during prediction. Therefore, memory requirements can become high for large datasets.

CBL also requires a case repository, but it can use case selection, indexing, and removal of irrelevant cases to manage memory more intelligently.

Prediction Strategy

KNN uses a fixed mathematical rule:

CBL follows a more flexible cycle:

Retrieve → Reuse → Revise → Retain

This allows CBL to modify an old solution when the new problem is not exactly the same.

Conclusion

KNN is simpler and effective when numerical features and a suitable distance measure are available. CBL is more suitable for medical diagnosis and expert systems where previous cases contain valuable knowledge and explanations. Thus, KNN emphasizes nearest-neighbour voting, whereas CBL emphasizes retrieval and adaptation of previous experiences.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Introduction

Both Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks can be used for function approximation. They give greater importance to data points that are close to the query or basis centre, but they work differently.

Consider the following dataset:

x	y
1	2.1
2	4.0
3	6.2
4	7.8
5	10.1
6	11.9

Suppose we want to predict y when x = 3.5.

Comparison

Feature	Locally Weighted Regression	RBF Network
Main idea	Fits a local regression model around query point	Uses radial basis functions as hidden units
Locality	Based directly on query point	Based on distance from basis centres
Training	Very little/no global training	Requires training of centres, widths and weights
Prediction cost	High because local model may be calculated for each query	Generally faster after training
Memory	Stores training data	Stores centres, widths and weights
Smoothness	Depends on bandwidth	Usually produces smooth predictions
Adaptation	Excellent local adaptation	Depends on selected basis functions
Scalability	Poor for very large datasets	Better during prediction

Locally Weighted Regression

LWR assigns weights according to the distance from the query point.

A commonly used weight function is: $$w_i = e^{-\frac{(x_i - x_q)^2}{2\tau^2}}$$

where:

- xq= query point
- xi= training point
- τ= bandwidth

For , points near 3.5 receive larger weights than points farther away.

Therefore, values such as:

- x = 3
- x = 4

will strongly influence the prediction, while x = 1 and x = 6 will have less influence.

RBF Network

An RBF network normally contains:

Input Layer → RBF Hidden Layer → Output Layer

The Gaussian radial basis function is: $$\phi(x) = e^{-\frac{||x-c||^2}{2\sigma^2}}$$

where:

- C= centre of the RBF
- σ= width

The final output is: $$y = \sum_{j=1}^{m} w_j \phi_j(x)$$

For , RBF units whose centres are near 3.5 produce larger activation.

Locality Difference

LWR is query-dependent. A new regression model is effectively formed around every query point.

RBF networks are centre-dependent. The network has predefined or learned centres, and the same basis functions are used for different queries.

Computational Cost

LWR can be computationally expensive during prediction because it must calculate weights and solve a local regression problem for each query.

RBF networks require more computation during training to determine centres, widths and output weights. However, once trained, prediction is generally faster.

Therefore:

LWR → expensive prediction, flexible local fitting

RBF → expensive training, efficient prediction

Smoothness of Predictions

RBF networks generally provide smooth predictions because radial basis functions overlap and their outputs are combined.

LWR can also produce smooth results, but smoothness strongly depends on the bandwidth parameter .

- Small bandwidth → highly local, less smooth

- Large bandwidth → smoother, more global

Conclusion

LWR is preferable when the dataset requires strong local adaptation and the number of queries is small. RBF networks are preferable when many predictions are required because the network can be trained once and then used efficiently. Thus, LWR provides flexible query-based locality, while RBF provides learned and smooth function approximation.

3. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

3. Naïve Bayes vs Logistic Regression for Spam Detection

Introduction

Consider a spam email classification system. Each email is represented using features such as:

- "free"
- "offer"
- "win"
- "discount"
- "prize"
- Number of links
- Number of capital letters

The target classes are:

Spam and Not Spam (Ham).

Both Naïve Bayes and Logistic Regression are popular methods for this task.

Comparison

Feature	**Naïve Bayes**	**Logistic Regression**
Type	Probabilistic classifier	Discriminative classifier
Main assumption	Features are conditionally independent	No independence assumption

Feature	Naïve Bayes	Logistic Regression
Probability	Directly estimates class probabilities	Estimates probability using sigmoid/softmax
Training	Very fast	Fast but iterative optimization is usually required
High-dimensional text	Very effective	Also highly effective
Data requirement	Works well with relatively small data	Usually benefits from more training data
Interpretability	Easy to understand	Coefficients can be interpreted
Correlated features	Can be affected	Handles correlation better
Common text model	Multinomial Naïve Bayes	Logistic Regression with TF-IDF

Naïve Bayes

Naïve Bayes uses Bayes' theorem:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

The classifier assumes that features are conditionally independent:

$$P(X|C) = \prod_i P(x_i|C)$$

For example, suppose an email contains:

"Win free prize now"

The classifier calculates:

P(Spam|Email)

And

P(Ham|Email)

If the first probability is larger, the email is classified as spam.

Logistic Regression

Logistic Regression calculates the probability of a class using:

$$P(y=1|x) = \frac{1}{1+e^{-z}}$$

where:

$$z = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

For text classification, words are converted into numerical features using methods such as Bag-of-Words or TF-IDF.

The learned weights determine how strongly each word contributes to spam classification.

Feature Independence

The major limitation of Naïve Bayes is its assumption that features are independent given the class.

For example:

"free" and "prize"

may occur together frequently in spam messages, so their occurrence is not truly independent.

Naïve Bayes still works surprisingly well despite this simplified assumption.

Logistic Regression does not require conditional independence, so it can better represent relationships among features.

Probability Estimation

Naïve Bayes estimates:

P(word|spam)

And

P(word|ham)

and combines these probabilities.

Logistic Regression learns weights for features and converts their weighted sum into a probability.

Computational Efficiency

Naïve Bayes is extremely fast because training mainly involves counting occurrences and calculating probabilities.

Logistic Regression requires optimization to learn weights, so training is generally more computationally demanding.

For very large text datasets, both can be efficient, but Naïve Bayes is usually faster to train.

High-Dimensional Data

Text data often contains thousands or millions of features.

Naïve Bayes performs very well with sparse, high-dimensional text data and can work effectively even with relatively small datasets.

Logistic Regression also performs strongly, particularly when sufficient labelled data and appropriate regularization are available.

When One Outperforms the Other

Naïve Bayes is preferable when:

- Training data is limited.
- Very fast training is required.
- Features are sparse.
- A simple probabilistic model is sufficient.
- A baseline classifier is needed.

Logistic Regression is preferable when:

- There are sufficient training examples.
- Feature relationships are important.
- High classification accuracy is required.
- Feature weights need to be interpreted.
- Regularization is required to control overfitting.

Conclusion

For spam detection, Naïve Bayes is usually faster and works exceptionally well with sparse high-dimensional text, while Logistic Regression can provide better decision boundaries when feature interactions and sufficient training data are available. Therefore, the choice depends on the dataset size, feature relationships, computational requirements, and desired performance.

4. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.

Introduction

Consider a dataset containing two classes of objects based on two features:

- Feature
- Feature

Suppose we want to classify a new point into Class A or Class B.

KNN and Support Vector Machine (SVM) use very different approaches.

Comparison

Feature	**KNN**	**SVM**
Learning type	Instance-based/lazy learning	Model-based learning
Training	Almost none	Requires model training
Prediction	Uses nearest training points	Uses learned decision boundary
Decision boundary	Implicit and local	Explicit and global
Scalability	Prediction becomes expensive for large datasets	Generally better after training
Noise sensitivity	Sensitive to noisy neighbours	Relatively robust with suitable margin
Memory	Stores training data	Stores support vectors/model parameters
Non-linear classification	Depends on distance/features	Kernel functions provide powerful non-linear boundaries

KNN Decision Boundary

KNN classifies a new point by finding its nearest neighbours.

The Euclidean distance is:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

For example, if:

$$K = 5$$

and the five nearest points contain:

- **3 Class A**
- **2 Class B**

then:

Prediction = Class A

The decision boundary is created implicitly from the distribution of nearby training points.

SVM Decision Boundary

SVM attempts to find the best separating hyperplane.

For a linear SVM: $wTx+b=0$

The optimal boundary maximizes the margin between the two classes.

The points closest to the boundary are called support vectors.

A larger margin generally provides better generalization.

Scalability

KNN has almost no training cost, but prediction can be expensive because distances may need to be calculated against many training samples.

For a large dataset, this can make KNN slow.

SVM has a higher training cost, especially for large datasets, but prediction is generally faster because it uses the learned model and support vectors.

Therefore:

KNN → low training cost, high prediction cost

SVM → higher training cost, lower prediction cost

Sensitivity to Noise

KNN can be sensitive to noisy data.

For example, if a noisy training point happens to be close to the test point, it may affect the majority vote.

Increasing can reduce the effect of individual noisy points, but an excessively large K can oversmooth the decision.

SVM focuses on maximizing the margin and can be more robust to individual noisy points. However, excessive noise and outliers can still affect the support vectors and decision boundary.

Kernel Functions in SVM

A major advantage of SVM is the use of kernel functions.

Common kernels include:

Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

Polynomial Kernel

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

RBF Kernel

$$K(x_i, x_j) = e^{-\gamma ||x_i - x_j||^2}$$

The RBF kernel can create a non-linear decision boundary.

For example, if Class A points are located inside a circular region and Class B points are outside it, a linear boundary cannot separate them effectively.

An RBF kernel can transform the problem into a higher-dimensional feature space where the classes can be separated.

Example

Suppose a dataset contains:

- Class A → points near the centre of a circle
- Class B → points surrounding the centre

KNN

KNN can classify points successfully based on their local neighbourhood.

Linear SVM

A linear SVM may fail because one straight line cannot separate the circular classes.

RBF-SVM

An RBF kernel can create a curved decision boundary and separate the two classes effectively.

Conclusion

KNN is simple and useful for small datasets where local similarity is meaningful. SVM is more suitable when a strong decision boundary is required, especially for high-dimensional data. Kernel functions make SVM particularly powerful for non-linear classification, while KNN relies directly on local neighbourhood structure.

5. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

Introduction

Clustering is an unsupervised learning technique used to group similar data points without predefined class labels.

Suppose a customer dataset contains:

- Age
- Annual income
- Spending score

The dataset has no known cluster labels. We want to discover groups of customers with similar characteristics.

Two common techniques are:

1. K-Means Clustering
2. Hierarchical Clustering

Comparison

Feature	**K-Means**	**Hierarchical Clustering**
Learning type	Unsupervised	Unsupervised
Cluster formation	Partitions data into K clusters	Builds a hierarchy of clusters
Number of clusters	Must usually be specified beforehand	Can be selected from dendrogram
Output	Flat clusters	Hierarchical tree/dendrogram
Computational cost	Generally lower	Generally higher
Scalability	Good for large datasets	Less suitable for very large datasets
Interpretability	Moderate	High due to dendrogram

Feature	K-Means	Hierarchical Clustering
Cluster shapes	Best for roughly spherical/compact clusters	Can represent more complex structures depending on linkage
Reassignment	Points can change clusters during iterations	Merges/splits are generally not reversed in standard approaches

K-Means Clustering

K-Means divides the dataset into clusters.

Algorithm

1. Select cluster centres.
2. Assign each point to its nearest centre.
3. Recalculate cluster centres.
4. Repeat assignment and updating.
5. Stop when cluster assignments or centroids become stable.

The objective is to minimize within-cluster squared distance:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

where:

- **Ck= cluster**
- **μk= centroid of cluster**

Example

Suppose customer data naturally contains:

- Low-income, low-spending customers
- Medium-income, medium-spending customers
- High-income, high-spending customers

If:

K = 3

K-Means attempts to create three groups around three centroids.

Hierarchical Clustering

Hierarchical clustering creates a tree-like structure called a dendrogram.

There are two major approaches:

Agglomerative

Starts with every data point as an individual cluster and repeatedly merges the closest clusters.

Divisive

Starts with one large cluster and repeatedly divides it.

Agglomerative clustering is more commonly used.

Cluster Distance

Different linkage methods can be used:

- **Single linkage –** minimum distance between points
- **Complete linkage –** maximum distance
- **Average linkage –** average distance
- **Ward linkage –** minimizes increase in within-cluster variance

Example

Suppose the data contains customer groups whose structure is unknown.

Hierarchical clustering initially treats each customer separately and gradually merges similar customers.

The resulting dendrogram allows us to select different numbers of clusters by cutting the tree at different levels.

Impact of Choosing Number of Clusters

K-Means

The number must normally be selected before clustering.

A poor value of K can produce poor clusters.

For example:

- K too small → different groups may be merged.
- K too large → one natural group may be divided unnecessarily.

Methods such as the Elbow Method and Silhouette Score can help choose K.

Hierarchical Clustering

The number of clusters does not have to be fixed at the beginning.

A dendrogram can be examined and cut at an appropriate level.

This makes hierarchical clustering useful when the underlying structure is unknown.

Computational Cost

K-Means is generally computationally efficient and scalable to large datasets.

Its computational cost is approximately related to:

$O(nKdi)$

where:

- **n=** number of data points
- **k=** number of clusters
- **d=** number of features
- **i=** number of iterations

Hierarchical clustering generally requires substantially more computation and memory because distances between clusters or data points must be maintained.

Therefore:

K-Means → better for large datasets

Hierarchical → better for smaller datasets requiring detailed structure

Handling Different Cluster Shapes

K-Means works best when clusters are:

- Compact
- Approximately spherical
- Similar in size

It can perform poorly when clusters have unusual shapes such as elongated or curved structures.

Hierarchical clustering can sometimes capture more complex structures depending on the distance measure and linkage method.

However, hierarchical clustering is not automatically better for every irregular shape; the selected linkage method strongly affects the result.

Interpretability

K-Means provides a centroid for each cluster, making the clusters easy to summarize.

However, hierarchical clustering has an important advantage: the dendrogram visually shows relationships among clusters at different levels.

Therefore, hierarchical clustering is often more interpretable when the goal is to understand the structure of the dataset.

Conclusion

K-Means is fast, scalable, and suitable for large datasets when the number of clusters is known or can be estimated. Hierarchical clustering is more computationally expensive but provides a detailed hierarchy of relationships and does not require the number of clusters to be fixed initially.

Therefore:

- Large dataset + speed → K-Means
- Unknown structure + interpretability → Hierarchical Clustering
- Clearly spherical clusters → K-Means
- Need to explore cluster hierarchy → Hierarchical Clustering